

ENGINEERING TAKE-HOME

Building a Document-Outline Agent

Marvin, the AI agent that builds and edits presentation decks

Hi Venkat,

Some background on what we do. Marvin is an AI agent that builds and edits presentation decks. A consultant describes what they want in plain English, and Marvin writes, restructures, and redesigns the slides. Most of our engineering work is in the agent layer: designing tools a language model can use, getting it to ask questions instead of guessing, and streaming what it's doing back to the user so the app feels responsive.

This is a smaller version of that.

There's no deadline. Do it whenever works for you, over a weekend if that's easier, and send it when you're happy with it. Keep the scope small. If you stop with something unfinished, write down what's missing and how you'd have done it. A small thing you understand well is worth more to us than something large you can't defend.

What you're building

A small web app where someone edits a document outline by talking to an agent. Two panels: the outline on the left, the chat on the right. The user types what they want in plain English, the agent does it, and the outline updates.

That's the whole product. It's small on purpose. The interesting part isn't the feature list, it's the decisions underneath it.

The stack

TypeScript, strict mode on. For the agent, use `langgraph deepagents` version `1.10.x`. The entry point is `createDeepAgent({ model, tools, systemPrompt, checkpointers })`. This is the same library and roughly the same setup we run in production, which is why we're pinning it rather than leaving it open. Tool schemas should use `Zod`.

For the backend we prefer NestJS, since that's what we run. Express is fine if you'd rather not pick up a new framework and a new library at the same time. We won't hold that against you. Frontend is React + Vite, one page.

Storage is a JSON file on disk. No database. Please don't add one. It's not the point and it'll cost you time.

For the model, use whatever provider you have access to. Keep the choice behind a single env var or config module, and don't hard-code keys or the provider, so we can drop in our own vars and run it locally to test when we review it.

The data

`outline.json` ships with this:

```
{
  "items": [
    { "id": "a1", "title": "Introduction", "description": "Set context and agenda." },
    { "id": "b2", "title": "Market Landscape", "description": "Size, growth, key segments." },
    { "id": "c3", "title": "Pricing Overview", "description": "Headline pricing model." },
    { "id": "d4", "title": "Competitive Analysis", "description": "How we compare to alternatives." },
    { "id": "e5", "title": "Pricing Details", "description": "Tiers, discounts, terms." },
    { "id": "f6", "title": "Next Steps", "description": "Owners and timeline." }
  ]
}
```

Array order is outline order. Position 1 is the first item. That's the whole data model.

The tools

Six tools. The names and shapes are yours to finalise. What matters is the capability.

`list-outline` takes no params and returns the current outline with positions.

`create-outline` takes a topic and an optional item count, and generates a fresh outline from scratch, replacing whatever's there. This one calls a model itself: the items are generated, not copied from anywhere.

`add-item` takes a title, an optional description, and an optional position, and inserts a new item.

`update-item` takes an item ID plus an optional new title and/or description, and edits the item.

`move-item` takes an item ID and a target position, and moves the item there.

`delete-item` takes an array of item IDs and removes them.

Six is the spec. Don't add a seventh.

What it needs to do

Changes should persist to `outline.json`. The outline panel should always reflect current state: when the agent changes something, the panel updates. How you do that is your call.

The chat should stream. The user needs to see the agent working while it works, tool activity at minimum and tokens if you can. A spinner that blocks until the whole turn resolves doesn't meet the bar. It's a big part of what makes the agent feel responsive rather than broken.

Conversation should be multi-turn and stateful. The agent remembers what was said earlier. If it asks the user something, they can answer in their next message and it carries on from there.

Natural language in. Users don't know item IDs and will never type one. Every instruction arrives as a sentence a person would actually say. Getting from that sentence to the right tool call with

the right arguments is the core of this exercise, and where most of your thinking should go.

Run these

Start from a fresh `outline.json`, open the app, and run these eleven prompts in order, in one session. Answer anything the agent asks the way a normal user would, then keep going down the list.

1. *What's in my outline?*
2. *Move the intro to the end.*
3. *Delete the pricing slide.*
4. *Move Pricing Details right after the Introduction.*
5. *Rename the last item to "Closing".*
6. *Change the description of Next Steps to "Owners, timeline, and budget sign-off."*
7. *Add a slide about implementation risks before Next Steps.*
8. *Move Competitive Analysis to the top and rename it to "Competitive Position".*
9. *Delete Market Landscape and Next Steps.*
10. *Move the appendix to the top.*
11. *Start over with a new outline about our Q3 expansion into Southeast Asia.*

These aren't a formality, and they aren't all as simple as they look. We'll run them against your checkout too.

What to send us

- A git repo we can clone and read. GitHub is easiest.
- A README.md covering how to run it from a clean checkout, the design decisions you made and why, and what you'd do with more time. The middle part is what we read most closely. If you considered an approach and dropped it, tell us. That's more interesting to us than the code you shipped.
- A TRANSCRIPT.md with the eleven exchanges above, pasted from the UI.
- A screen recording, three or four minutes, of you running all eleven. Unedited. If something breaks on camera, leave it in. We'd rather see the real thing than a clean take.
- `npm install && npm run dev` (or your equivalent) working from a clean checkout. If there's a setup step, put it in the README.

What we're looking at

A take-home with unstated criteria is a guessing game, so here's roughly where our attention goes.

Judgment, most of all. Every one of those eleven prompts involves a decision about what the agent should do when the instruction isn't clean, and there's usually more than one defensible

answer. We care which one you picked and whether you thought about it, not whether it matches what we'd have done.

How you shape tools for the model. Schemas, descriptions, what you return and when. The model can't read your code, only what you describe in the tool definitions, and it'll call the wrong tool if the descriptions are loose. Designing for that reader is the main skill here.

What happens when things go wrong. Bad input, missing items, instructions that don't map onto anything.

Whether you finished what you started. We'd rather see a smaller part that works than a bigger one that doesn't.

We're not scoring you on visual design, test coverage, commit hygiene, performance, or how many of the eleven you got perfect.

Good luck. We're looking forward to what you make of it.

The Marvin team

 StrategyConnect · strategyconnect.co